Z7i Exam App

Software Architecture - Handover document - 21 October 2026

1. Overview

Z7i Exam App (launcher label "Z Exam") is a Flutter, tablet-first app locked to landscape. Students log in with their enrolment number, see their exam list, download a password-protected paper, write it in a full-screen player that pins the app (Android lock task) and submit it; afterwards they see an analysis or, once published, the full result with solutions. It uses GetX for state **and** navigation, get_it for service lookup and a single Dio-based `ApiService`.

Area	Choice
Framework	Flutter stable, Dart SDK ^3.10.1 (`pubspec.yaml`)
App version	1.0.7+4 in `pubspec.yaml`; the header shows `AppConstants.appVersion` = 1.0.6
Platform	Android tablets (landscape), usually under MDM
State management	GetX (`get` ^4.6.6): GetxController, Rx observables, Obx and GetBuilder
Navigation	GetX named routes (`GetMaterialApp` + `GetPage`). go_router ^14.2.7 is a dependency but not used
Dependency lookup	get_it ^7.7.0 via `ServiceLocator`
HTTP	dio ^5.4.0 (all API calls, image downloads, reachability probe)
Local storage	get_storage ^2.1.1 behind `StorageService`
Connectivity	connectivity_plus ^5.0.2
Maths / HTML	flutter_tex ^5.1.10 (TeX rendering server + TeXView); flutter_math_fork and latext are listed but effectively unused
Web content	webview_flutter ^4.13.1 (exam instructions)
Device management	managed_configurations ^1.0.2 (MDM), native lock task via a MethodChannel
Other	intl ^0.18.0 (date formatting on the exam list)

2. Getting started

You need a Flutter stable SDK that ships Dart 3.10 or newer, and Android Studio with JDK 17 and the Android SDK.

1. `flutter pub get`
2. Run on a landscape tablet or emulator: `flutter run`
3. Release build: `flutter build apk --release`

Setting	Where	Value today
API base URL	`lib/core/constants/app_constants.dart` (`AppConstants.baseUrl`)	`https://portalapi.z7i.in/api/crm`
Displayed version	`AppConstants.appVersion`	1.0.6 (bump by hand with pubspec.yaml)
Android application ID	`android/app/build.gradle.kts`	`com.example.suguna_exam_app`
Android release signing	`android/app/build.gradle.kts`	Release uses the debug signing config (TODO in the file)

There are no build flavours or environment files. Three alternative `baseUrl` values (a public IP and two LAN IPs on port 3012) are commented out in `AppConstants`; swap the lines to point at another backend.

To test the MDM path without a managed device, log in normally: the login screen stores the response, and `LoginController.checkLogin()` skips the login screen on the next start.

3. Project structure

All app code (77 Dart files) lives in `lib/`. Shared code is in `lib/core/`, models and local services in `lib/data/`, and each screen family is a folder under `lib/modules/`.

Path	What it holds
`lib/main.dart`	Startup, MDM silent login, `MyApp` (`GetMaterialApp`)
`lib/core/routes/`	`app_routes.dart` (16 path constants), `route_config.dart` (the GetPage list)
`lib/core/services/`	`ApiService` (Dio base), `AssignmentApiServices` (every endpoint), `service_locator.dart`
`lib/core/constants/`	`AppConstants`: base URL, timeout, version, layout sizes
`lib/core/widgets/`	`AppKeyboard` / `KeyboardInputBox` (in-app keyboard), unused AppDrawer, AppAppBar, AppRichText, ScoreCircle
`lib/core/theme/, utils/, resources/`	`AppColors`, `AppTheme` (light only), `ResponsiveHelper`, `AppStrings`
`lib/data/models/`	ExamModel, QuestionModel
`lib/data/services/`	`StorageService` (GetStorage), `Session` (in-memory user), `NetworkService` (connectivity + reachability), `MockDataService`
`lib/modules/<feature>/`	One folder per feature (see section 6)
`android/app/src/main/kotlin/../MainActivity.kt`	The "exam_lock" MethodChannel: lock task, hidden system bars, FLAG_SECURE, force exit
`assets/`	`test.json` (sample paper, loader is never called)

Most feature folders follow the same three-part GetX layout:

- **bindings/**: a Bindings class that `Get.lazyPuts` the controller. Each `GetPage` names its binding.
- **controllers/**: a GetxController that loads data in `onInit()` (or `onReady()`) through `AssignmentApiServices` and exposes .obs fields.
- **views/**: widgets that read the controller with `GetView`, `Get.find` or `GetBuilder` and rebuild through Obx / update().

Exceptions: the exam list view is `discussion_list/views/download_test.dart` (class DownloadTest); `lib/modules/download_test/` is an empty folder.

4. Architecture & patterns

Every screen follows one path: GetX runs the route binding, the binding creates the controller, the controller calls AssignmentApiServices, and that sends a POST through the shared Dio client. The logged-in user lives in Session (memory) and StorageService (disk); downloaded papers, pending submissions and caches also live in StorageService. Only the test player talks to native code.

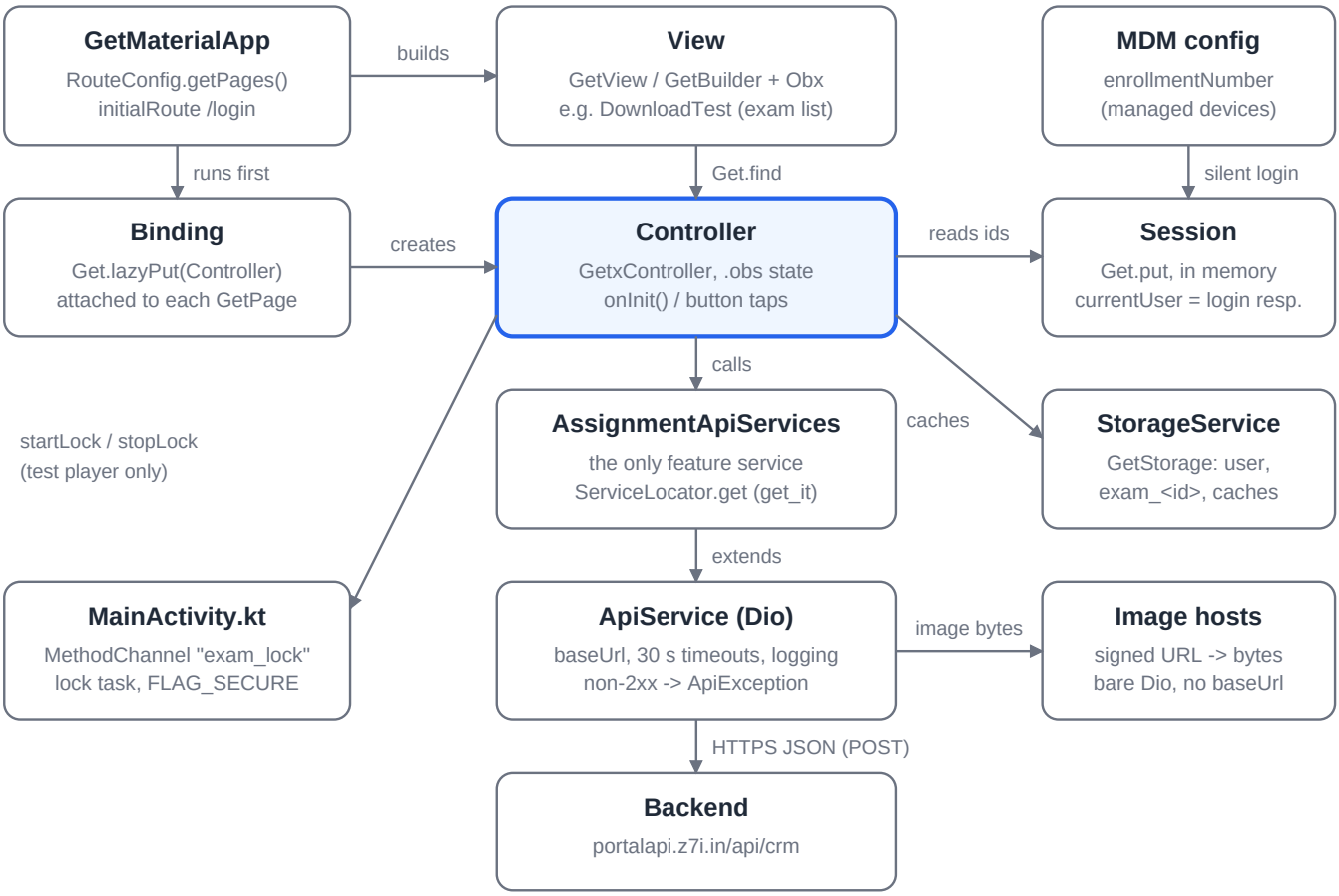

Figure 1. Request path for a typical screen, with routing, session, storage, MDM and the native exam lock around it.

Startup sequence (lib/main.dart)

1. `WidgetsFlutterBinding.ensureInitialized()`
2. Start the TeX rendering server (`TeXRenderingServer.start()`), retrying up to 3 times 500 ms apart.
3. Lock orientation to landscape left/right.
4. `Get.put(StorageService(), permanent: true)` and `Get.put(Session())`.
5. `GetStorage.init()`: opens the storage box.
6. `ServiceLocator.setupServices()`: registers ApiService and AssignmentApiServices as get_it lazy singletons.
7. MDM silent login: read `enrollmentNumber` from managed configuration; if set and no user is stored, call `login` and save the response to Session and storage. Errors are only logged.
8. `runApp(MyApp())`: `GetMaterialApp` with `AppTheme.lightTheme`, `initialRoute: /login`, `getPages: RouteConfig.getPages()`.
9. On the login screen, `LoginController.onReady()` calls `checkLogin()`: if a user is stored it goes straight to `/discussion` (the exam list).

State management (GetX)

Controllers hold state as Rx fields (`isLoading = false.obs`, `exams = <ExamModel>[].obs`). Views wrap the changing parts in `Obx`. `TestPlayerController` mixes in manual `GetBuilder` updates with ids (`update(['palette'])`, `update(['new'])`). `Session` is a GetxController used as a global bag of mutable fields (currentUser, currentTestId, isNewTest, isRetest, storedData, selectedSubject). The player runs in exam mode when `Session.currentTestId` starts with `exam_`.

Navigation (GetX routes)

`RouteConfig.getPages()` lists 16 GetPages. Navigation uses `Get.toNamed`, `Get.offNamed` and `Get.offAllNamed`; data is passed in `arguments` (a Map with examId, examCode, examName and, for the player, the whole paper under `selectedQuestion`). There is no auth guard or redirect: the login check is done by `LoginController.checkLogin()`. The live flow is /login -> /discussion -> /assignment-instructions -> /test-player -> /exam-analysis, with /exam-result, /exam-question-review and /absent-exam-questions for results. Back is intercepted with `PopScope` on the player, instructions and result screens.

Dependency lookup (get_it)

`ServiceLocator` wraps `GetIt.instance`. It registers `ApiService` and `AssignmentApiServices` as lazy singletons; every controller uses `ServiceLocator.get<AssignmentApiServices>()`. StorageService and Session are registered in GetX instead (`Get.find`), and a few classes also create `StorageService()` directly (LoginController); that works because they share the same GetStorage box.

5. Networking & services

Every backend call goes through AssignmentApiServices, a subclass of ApiService. Requests carry no auth header: the backend identifies the student by tenantId / entiId and admissionNo / regNo / studentId sent in each body. All endpoints are POST.

ApiService (base)

- Dio with `AppConstants.baseUrl`, 30 s connect/receive timeouts, JSON Content-Type/Accept headers and a `LogInterceptor` that logs full request and response bodies (also in release).
- Only `get` and `post` exist. They return Map<String, dynamic>; an empty body becomes `{success: true}` and a list body becomes `{success: true, data: [...]}`.
- Dio errors become `ApiException(message, statusCode)` with the response's `message` field or "An error occurred".
- `downloadImageBytes(url)` uses a fresh Dio (no baseUrl, no JSON headers) to fetch raw image bytes from the storage host.
- Endpoints answer `{status: true, data: ...}`; controllers check `response['status'] == true`.

Session and login

- `login(regNo)` posts `{regNo}` to /student/login. There is no password; the login screen only has a digit keypad for the enrolment number.
- On `status == true` the whole response is stored as `Session.currentUser` and in GetStorage key `user`. Other calls read `currentUser['data']`: tenantId, admissionNo, studentId, batchCode, batchId, centerId, classNo, name.
- "Logged in" means the `user` key exists (`StorageService.isLoggedIn`). Nothing in the app logs out: `clearUser()` is never called.

Offline storage (GetStorage keys)

Key	Written by	Holds
`user`	Login, MDM login	Full login response
`exam_list_cache_<admissionNo>`	Exam list	Last good exam list, shown offline
`exam_<examId>`	Exam download, player submit	Paper with images embedded as base64; later completed / pendingSync / examPayload
`active_exam_id`	Exam download	Exam currently in progress (blocks starting another non-live exam)

AssignmentApiServices: methods and endpoints

All paths are relative to `https://portalapi.z7i.in/api/crm`.

Method	Endpoint	Body keys
login	/student/login	regNo
getExamList	/exam/student-list	entiId, batchId, admissionNo
getExamQuestions	/exam/questions/get-questions	tenantId, examId, examCode, password
downloadExamQuestionImage	/exam/questions/image/download	url
submitExamResult	/exam/result/submit	payload map (see section 7)
getExamAnalysis	/exam/result/my-result/analysis	entiId, examId, admissionNo
getExamResultWithQuestions	/exam/result/my-result/with/questions	entiId, examId, admissionNo
getAbsentExamQuestions	/exam/questions/get-questions/absent	tenantId, examId, examCode
reportQuestion	/assignment/question/report/create	payload map

Error style differs by method. `submitExamResult` catches errors and *returns* the exception; the report and image methods let ApiException propagate; the rest rethrow. Callers check `response is Exception` in places where that can never be true.

Images. Papers and results are made to work offline by replacing every `<img src>` in question, passage, option, match-column and solution HTML (and the `images[]` list) with a base64 data URI: ask the backend for a

signed URL, then fetch the bytes with `downloadImageBytes`, falling back to the original URL. Exam downloads retry each image once and refuse to cache a paper if any image failed.

Connectivity. `NetworkService.hasInternet()` checks only that a network interface is up. `canReachServer()` also sends a GET to the base URL (any status counts) and is used before an exam submission.

6. Feature modules

Students move through these modules in order: login, exam list, instructions, player, then analysis or result.

Module	Route(s)	What it does
auth	/login	Enrolment number on an in-app digit keypad; auto-skips if a user is stored
discussion_list	/discussion	Exam list (DownloadTest): tabs by age, status chips, Test ID + passcode download panel, refresh, exit app
assignment_instructions	/assignment-instructions	Shows instructions HTML in a WebView; Accept opens the player. Also reused read-only from the player
test_player	/test-player	Full-screen exam player: timer, calculator, question palette, report question, submit
exam_result	/exam-analysis, /exam-result, /exam-question-review, /absent-exam-questions	Post-submit analysis, published result with solutions, per-question review, question paper for absentees

7. Screen-wise API calls

Every call below is a POST relative to `https://portalapi.z7i.in/api/crm` with a JSON body, unless noted. Body values come from the saved login (`Session.currentUser['data']`, loaded from GetStorage key `user`):

- `tenantId` / `entiId` = data.tenantId
- `admissionNo` / `regNo` = data.admissionNo
- `studentId` = data.studentId; `batchId` = data.batchCode

A successful response has the shape {status: true, data: ...}. Errors surface as a snackbar or an error panel with Retry.

Login *(auth/)*

When	Method + endpoint	Request body / params	Service method
Tap Login	POST /student/login	{regNo}: the enrolment number typed on the keypad	AssignmentApiServices .login
App start on an MDM device, before runApp	POST /student/login	{regNo}: enrollmentNumber from managed configuration	AssignmentApiServices .login

Exam list *(discussion_list/ (DownloadTest))*

When	Method + endpoint	Request body / params	Service method
Screen opens, and the header Refresh button (once per stored exam_* with pendingSync: true)	POST /exam/result/submit	The examPayload saved by the player when an earlier submit did not complete	AssignmentApiServices .submitExamResult
Screen opens, pull-to-refresh, Refresh, Retry	POST /exam/student-list	{entiId, batchId: batchCode, admissionNo}. On failure the cached list is shown	AssignmentApiServices .getExamList
Tap a card -> enter passcode -> Download & Start, or the Test ID + passcode panel	POST /exam/questions/get-questions	{tenantId, examId, examCode, password}: password is the passcode typed. Skipped if exam_<examId> is already cached	AssignmentApiServices .getExamQuestions
Right after the call above, once per image in questions and instructions	POST /exam/questions/image/download	{url}: image URL from the HTML. Returns a signed url	AssignmentApiServices .downloadExamQuestionImage
Right after each signed URL	GET <signed url> (outside the base URL, raw bytes)	none. Bytes are embedded as base64	ApiService .downloadImageBytes

Test player *(test_player/)*

When	Method + endpoint	Request body / params	Service method
Submit Test, Back, or Home/Recents pressed (opens the type-SUBMIT dialog), then Confirm	GET <baseUrl> (bare Dio, any status = reachable)	none. Checked before the dialog and again before sending	NetworkService .canReachServer
Confirm in the SUBMIT dialog	POST /exam/result/submit	{entiId, examId, admissionNo, studentId, batchId: batchCode, centerId, classNo, startedAt, completedAt, sectionTimings, answers[{questionId, sectionId, answerRaw, isSkipped, timeTaken ms}]}	AssignmentApiServices .submitExamResult
Tap Report, pick an issue, enter a comment, Submit	POST /assignment/question/report/create	{entiId, studentId, studentName: data.name, questionId, assignmentId: currentTestId, issueType: "WRONG_ANSWER", description}	AssignmentApiServices .reportQuestion

Exam analysis *(exam_result/ (/exam-analysis))*

When	Method + endpoint	Request body / params	Service method
Screen opens (after submit, or tapping an attempted exam) and Retry	POST /exam/result/my-result/analysis	{entiId, examId, admissionNo}: examId from route arguments	AssignmentApiServices .getExamAnalysis

Exam result *(exam_result/ (/exam-result))*

When	Method + endpoint	Request body / params	Service method
Screen opens (card with isResultVisible) and Retry	POST /exam/result/my-result/with/questions	{entiId, examId, admissionNo}. "Result not found" / "not visible yet" -> Absent questions screen	AssignmentApiServices .getExamResultWithQuestions
Right after the call above, once per image	POST /exam/questions/image/download, then GET <signed url>	{url}	downloadExamQuestionImage, downloadImageBytes

Absent exam questions *(exam_result/ (/absent-exam-questions))*

When	Method + endpoint	Request body / params	Service method
Screen opens (redirect from Exam result) and Retry	POST /exam/questions/get-questions/absent	{tenantId, examId, examCode}	AssignmentApiServices .getAbsentExamQuestions
Right after the call above, once per image	POST /exam/questions/image/download, then GET <signed url>	{url}	downloadExamQuestionImage, downloadImageBytes

No API calls: Exam instructions (WebView of stored HTML), Exam question review (uses ExamResultController data) and the player between download and submit (answers stay on the device).

Defined but not called: ApiService.get and TestApiParser.loadQuestionsJson (assets/test.json).

8. Platform specifics (Android)

The app is built for school tablets under MDM: landscape only, pinned while an exam runs, and optionally logged in by the device policy.

Concern	How it works	Where
Orientation	Landscape left/right at startup; manifest sets sensorLandscape	main.dart, AndroidManifest.xml
Exam lock	Player calls `startLock` when the timer starts: hides system bars, sets FLAG_SECURE and FLAG_KEEP_SCREEN_ON, calls startLockTask() and re-pins every 1.5 s if unpinned. `stopLock` reverses it shortly after the player route is left	MainActivity.kt, test_player_controller.dart
Home / Recents during an exam	Native sends `homePressed` (onUserLeaveHint, or focus lost while paused) and brings the task back to front; Flutter opens the SUBMIT dialog. A 4 s grace period after lock ignores spurious events	MainActivity.kt, test_player_view.dart
Exit app	Exam list Exit -> `exitApp`: stopLockTask with retries, finishAndRemoveTask, killProcess. Falls back to SystemNavigator.pop	download_test.dart, MainActivity.kt
Lock task mode	Activity has lockTaskMode="if_whitelisted"; full Home blocking needs the MDM to allow-list the app	AndroidManifest.xml
MDM config	managed_configurations reads enrollmentNumber once at startup (no live listener)	main.dart
Screenshots	Blocked only while the exam lock is on (FLAG_SECURE)	MainActivity.kt
Keyboard	In-app AppKeyboard for login, passcode, Test ID and SUBMIT so the OS keyboard does not cover the landscape screen	core/widgets/app_keyboard.dart
Maths rendering	TeXRenderingServer (local server + webview) started in main; questions render through flutter_tex	main.dart, test_player/views/
Theme	Light theme only	core/theme/

Android. applicationId and namespace com.example.suguna_exam_app; label "Z Exam"; Java/Kotlin target 17; min/target/compile SDK follow Flutter defaults. Permission: INTERNET only. usesCleartextTraffic="true". Release builds are signed with the debug key.

Native plugins (Android). connectivity_plus, managed_configurations, path_provider, url_launcher, webview_flutter (used by flutter_tex and the WebViews).